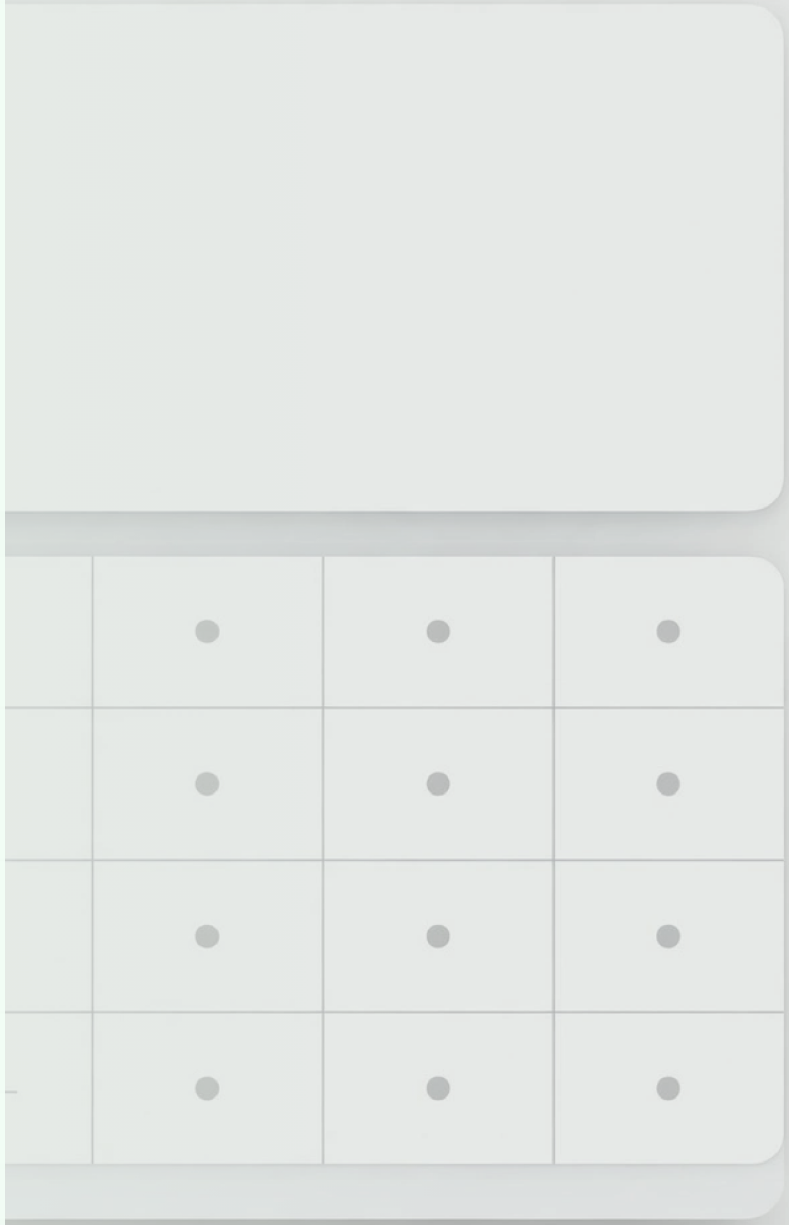
RETAIL GOVERNED ANALYTICS • PLAIN TEXT TO SQL

Text-to-SQL for Retail Governed Analytics Assistant

Frontend → API Gateway → Backend Orchestrator → LLM Gateway → Validator → Semantic Views → Warehouse → Explanation. Business user asks in plain text, no SQL knowledge needed, safe enterprise design.

Sales | Margin | Store | Inventory | Read-only | Audited



User experience – plain text to answer

BUSINESS USER ENTERS

“Show total sales and margin by store in Bangkok last week”

APPLICATION RETURNS

฿125.4M
Bangkok sales last week

12.8%
Gross margin

Top stores: Store A • Store B • Store C • No direct DB access, no SQL knowledge needed

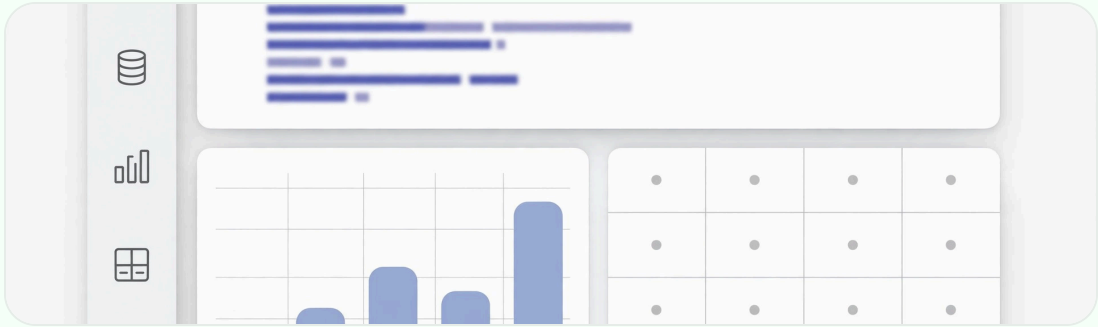
Answer + Evidence + Freshness + SQL if authorized — chart, table, SQL used, dataset used, freshness, download, follow-up box

Analytic Assistant • Fresh 08:00 ICT

Store A — Siam	**฿5.2M** · 13.1%
Store B — Sukhumvit	**฿4.8M** · 12.4%
Store C — Bangkapi	**฿4.1M** · 11.9%

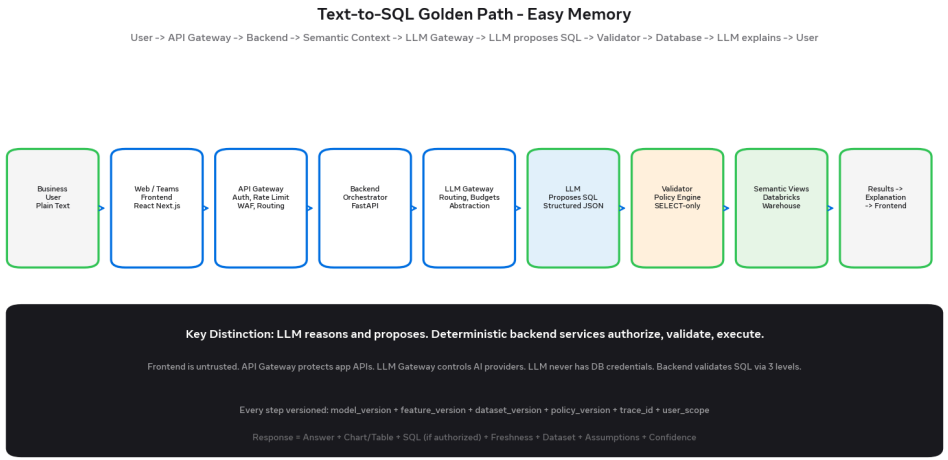
Chart • Table • SQL • Assumptions • Confidence 0.91 ↗

Key principle: user never writes SQL. LLM proposes, deterministic services validate. Trust comes from showing evidence, not just fluent answer.



Easy memory – the golden path

User → API Gateway → Backend → Semantic Context → LLM Gateway → LLM proposes SQL → Validator approves SQL → Database executes SQL → LLM explains result → User sees evidence



KEY DISTINCTION

LLM reasons and proposes. Deterministic backend services authorize, validate, execute. This separation makes Text-to-SQL safe for enterprise data. Frontend is untrusted. LLM never has DB credentials.

TRUST BOUNDARY

Every step versioned: **model_version + dataset_version + policy_version + trace_id + user_scope**
Response = **Answer + Chart/Table + SQL + Freshness + Dataset + Assumptions + Confidence**

Frontend – React / Next.js + TypeScript

WHAT IT DOES – LIMITED AND TRUSTED BOUNDARY

- **Authenticate** via corporate SSO (Entra ID), attach access token and conversation_id
- **Capture question** – plain text, response_mode: chart_and_table
- **Stream progress** via SSE or WebSocket, render table/chart, SQL if authorized
- **Show evidence**: freshness, dataset, assumptions, confidence, download, feedback

```
POST /v1/chat/query
{ "question": "Show sales by store in Bangkok last week", "conversation_id": "conv-789",
"response_mode": "chart_and_table" }
```

Stack: Next.js, TypeScript, Charting, SSO

WHAT IT DOES NOT DO – BACKEND PERSPECTIVE UNTRUSTED

- **Never** store DB credentials, warehouse tokens
- **Never** hold LLM API keys or system prompt
- **Never** execute SQL or decide authorization
- **Never** calculate business logic or cost
- **Never** bypass API Gateway – all via validated tokens

Frontend is untrusted. Safety relies on backend validation, not UI controls. If UI can be spoofed, backend still enforces SELECT-only + row-level.

FIRST BACKEND ENTRY • PROTECTS YOUR APP APIS

API Gateway – protects your application APIs

FRONTEND → GATEWAY → BACKEND

Web / Teams → **API GW** → Orchestrator

Frontend is untrusted. Gateway is policy enforcement point before any business logic. No SQL, no LLM key ever passes here.

HANDLES

- **Auth validation,** TLS termination, CORS, WAF
- **Rate limiting,** request-size limits, timeout controls
- **Versioning, routing,** trace_id / request_id injection
- **Basic audit logging** — who asked what, when

EXAMPLE ENDPOINTS

POST /v1/chat/query	→ async
GET /v1/query/{id}/status	SSE / WS
GET /v1/query/{id}/result	table+chart
POST /v1/feedback	thumbs up/down

IMPLEMENTATION OPTIONS

Azure APIM · **AWS API GW** · **Kong** · **Apigee** · **NGINX** · **Tencent GW**

Choose managed. Don't build auth/rate-limit yourself. Gateway never sees DB credentials or LLM prompts — only token + question reference.

Application backend – FastAPI orchestrator, the brain

Use FastAPI for AI orchestration, keep security deterministic. Never ask LLM to do everything in one prompt — use controlled state machine.

STACK CHOICE

Python FastAPI | Spring Boot alt | Pydantic validation | LangGraph optional

Security deterministic — parser, allowlist, Unity Catalog, not LLM

LLM is proposer only — backend authorizes, validates, executes

12 steps logged with trace_id — auditable, replayable

WHY NOT SINGLE PROMPT?

Single prompt = unsafe. Skips auth, sends full catalog, no validation, no cost control. State machine guarantees validator cannot be bypassed.

Orchestrator does 12 steps — deterministic

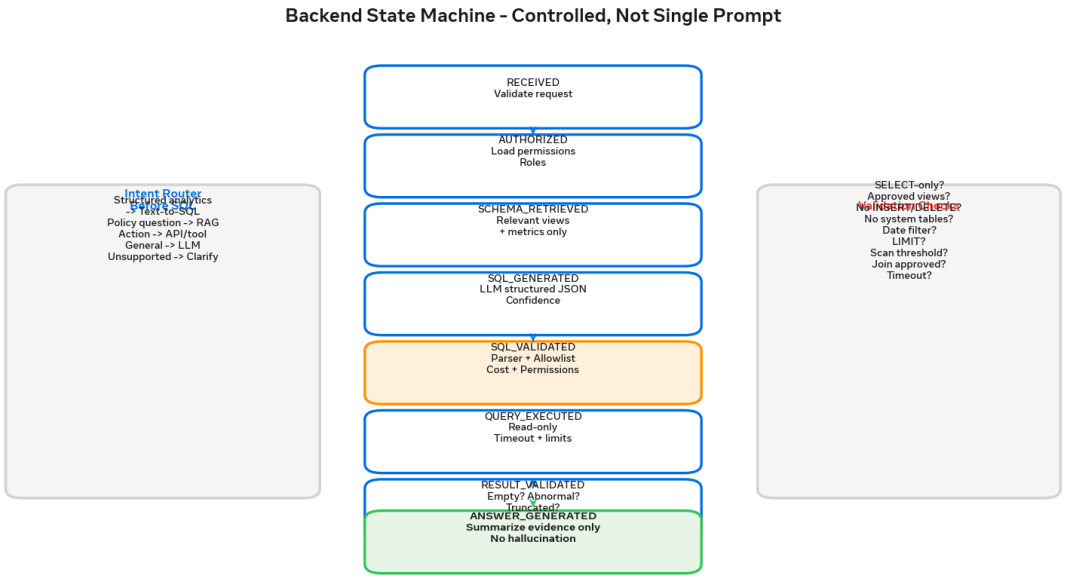
TRACE_ID • AUDIT

1. **Validate user & request** — SSO token, schema of question
2. **Load permissions** — Roles, row/col level, region
3. **Classify intent** — SQL vs RAG vs API vs clarify
4. **Retrieve semantic metadata** — Relevant views only, not full catalog
5. **Build controlled context** — Tables, metrics, joins, examples
6. **Request structured SQL plan** — JSON via LLM Gateway
7. **Validate generated SQL** — Parser + policy engine + cost
8. **Execute scoped read-only** — User OAuth, timeout, LIMIT
9. **Validate result** — Empty? Abnormal? Truncated?
10. **Summarize evidence only** — Rows + definitions → answer
11. **Return answer + evidence** — Chart, table, SQL, freshness
12. **Write audit events** — Trace, tokens, cost, policy decision

Each transition logged. If any validation fails → CLARIFICATION or ERROR with safe message. No credentials ever reach LLM.

State machine – RECEIVED to ANSWER

Don't ask LLM to do everything in single prompt. Use controlled state machine — each transition logged with trace_id. Prevents LLM skipping validation, ensures auditability.



WHY DETERMINISTIC?

RECEIVED → AUTHORIZED → SCHEMA_RETRIEVED → SQL_GENERATED → SQL_VALIDATED → QUERY_EXECUTED → RESULT_VALIDATED → ANSWER_GENERATED

If any validation fails → CLARIFICATION or ERROR with safe message. Optional LangGraph, but keep control in code.

INTENT FIRST

→ **Structured analytics** → Text-to-SQL, **Policy/doc** → RAG, **Action** → approved API, **Chat** → general LLM

Validation checks in center box: SELECT-only? Approved views? No system tables? Date filter? LIMIT? Cost threshold? Join approved? Timeout? Row access?

OBSERVABILITY

Each state emits: trace_id, latency, tokens, policy_decision, sql_hash, row_count, user_scope_version. Immutable audit trail.

Security gate — SQL_VALIDATED uses parser + allowlist + permission + cost + explain-plan. Orange state in diagram.

Hallucination firewall — RESULT_VALIDATED checks empty, abnormal, truncated before ANSWER_GENERATED summarizes evidence only.

L6 mindset — Technical completion ≠ business readiness. State machine forces checkpoints, not just one LLM call.

Intent router – classify first

Before generating SQL, route the question. Prevents LLM from doing RAG when it should do SQL, or doing SQL when it should do an approved API. Same rule: RAG for documents, Text-to-SQL for structured data, APIs for actions.

STRUCTURED ANALYTICS
Text-to-SQL

"Sales by store last week" → vw_sales_summary

→ CERTIFIED VIEWS

POLICY / DOCUMENT
RAG

"What is return policy?" → vector search over docs

→ KNOWLEDGE BASE

ACTION REQUEST
Approved API / Tool

"Update price to ฿99" → pricing API with approval

→ GOVERNED ACTION

GENERAL
Normal LLM

"Why did sales decline?" → SQL + controlled workflow

→ OBSERVATION ONLY

UNSUPPORTED
Clarification

Vague, ambiguous, missing scope → ask for region / date / metric

→ SAFE FALLBACK

Why this matters: Prevents one-prompt-does-all anti-pattern. Router uses smaller cheaper model. Only structured analytics goes to expensive SQL generation model with semantic context.

Implementation: LangGraph node or simple classifier prompt. Log intent + confidence. If confidence < 0.72 → clarification. Trace_id preserved across router → SQL gen.

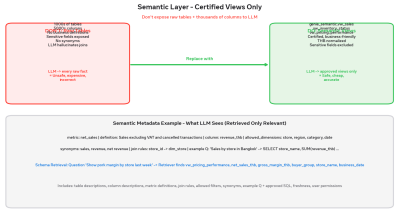
Semantic layer – certified views only

Don't expose raw tables + thousands of columns to LLM — replace with exclusive business-friendly interface

DON'T: RAW TABLES

- × 1000s tables, 5000+ columns, no definitions
- × Sensitive fields exposed, no synonyms
- × LLM hallucinates joins, expensive scans
- × Unsafe, expensive, incorrect

LLM → every raw fact = 3 failures



DO: CERTIFIED VIEWS

- ✓ genie_semantic.vw_sales_summary, vw_inventory_status, vw_pricing_performance
- ✓ Certified, business-friendly, THB normalized, standard dates, sensitive excluded
- ✓ last_updated field, row filters embedded, versioned
- ✓ LLM → approved views only = Safe, cheap, accurate

METRIC EXAMPLE

metric: net_sales
definition: Sales excl VAT & cancelled
column: revenue_thb
allowed_dims: store, region, category, date

WHAT LLM SEES

synonyms: sales, revenue
join: store_id → dim_store
example Q: 'Sales by store in Bangkok' → SELECT ...
retrieval: only relevant context

Schema retrieval: Question "Show pork margin by store last week" → Retriever finds vw_pricing_performance, net_sales_thb, gross_margin_thb, buyer_group. Includes table desc, column desc, metric defs, join rules, allowed filters, synonyms, approved SQL, freshness, permissions.

Schema retrieval – send only relevant context

Don't send complete catalog to model on every question — use retrieval. Vector search over schemas, metric definitions, approved SQL examples. Cheaper, safer, more accurate.

WHY RETRIEVAL, NOT FULL DUMP?

- **1000s tables + 5000 cols** = token bloat, hallucinated joins, sensitive leak, $ cost
- **RAG for tables:** question → retriever finds 2-3 views + 6-8 columns only
- **Scoped by permission** — user can only retrieve views they have access to
- **Freshness aware** — includes last_updated, so LLM can warn about stale

RETRIEVED CONTEXT INCLUDES

- Table & column descriptions
- Join rules (store_id → dim_store)
- Synonyms: sales, revenue, net
- Data freshness version
- Metric definitions (net_sales = excl VAT)
- Allowed filters + enums
- Example Q + approved SQL
- User permission scope

QUESTION

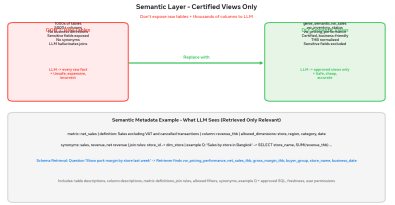
"Show pork margin by store last week"

↓ Retriever finds — relevant only

- vw_pricing_performance
- net_sales_thb
- gross_margin_thb
- buyer_group
- store_name
- business_date

Then send only that context to LLM — not 200 tables. LLM sees certified business definitions, not raw fact tables.

Pattern: Embedding model indexes metric registry + view DDL + approved queries. Cosine search at query time, re-rank by permission.



DIFFERENT JOBS — DON'T CONFUSE

LLM Gateway – controls access to AI providers

BOUNDARY

API Gateway

Protects YOUR app APIs

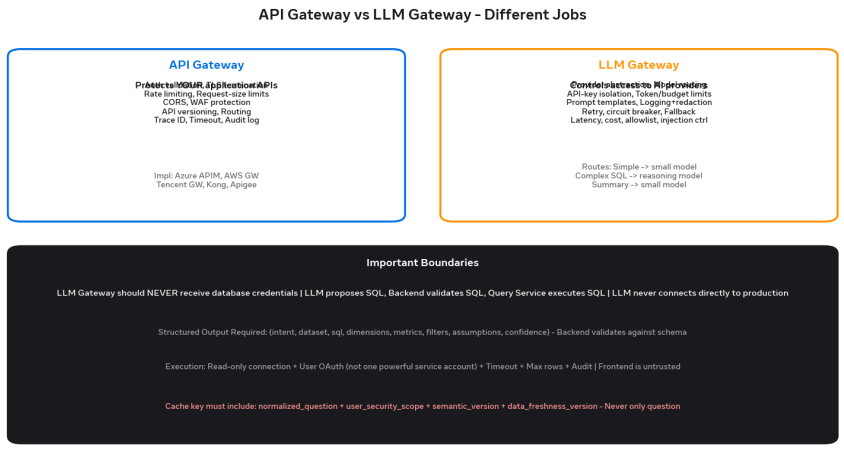
LLM Gateway

Controls AI providers

Architecture: Text-to-SQL Backend → LLM Gateway → Azure OpenAI / OpenAI / Gemini / DeepSeek / Qwen. Backend never gives DB creds to gateway.

GATEWAY PROVIDES

- Provider abstraction & routing
- Token + budget limits per user/team
- Retry + circuit breaker + fallback
- Model allowlist
- API-key isolation
- Prompt templates & redaction logging
- Latency, cost, token metrics
- Content & prompt-injection controls



ROUTING EXAMPLE

Simple intent classification	**Small cheap model**
Complex SQL generation	**Strong reasoning**
Result summarization	**Small model**
Sensitive workload	**Private regional endpoint**

Critical: LLM Gateway should NEVER receive database credentials. Flow: LLM proposes SQL → Backend validates SQL → Query Service executes SQL. LLM never connects directly to production.

Require structured JSON — validate before use

LLM OUTPUT CONTRACT — NOT FREE TEXT

Pydantic • JSON Schema

```
{
  "intent": "analytics_query",
  "dataset": "genie_semantic.vw_sales_summary",
  "sql": "SELECT store_name, SUM(revenue_thb) ..." ,
  "dimensions": ["store_name"],
  "metrics": ["revenue_thb"],
  "filters": [{
    "field": "region_name",
    "operator": "=",
    "value": "Bangkok"
  }],
  "assumptions": ["Sales means net sales excl VAT"],
  "confidence": 0.91
}
```

WHY STRUCTURED MATTERS

Backend validates object against schema before using — no mixed prose + SQL parsing

Assumptions must be shown to user — transparency builds trust

Confidence routing — if < 0.72 threshold → clarification, not execution

Auditable — dataset, dimensions, filters logged separately from SQL

Anti-pattern to avoid: LLM returns “Here is your SQL: SELECT ...” with explanation woven in. Backend extracts via regex → brittle, injectable.
Instead: System prompt says “Respond only with valid JSON matching schema. No markdown, no comments.”

IMPLEMENTATION

Use OpenAI function calling / structured output mode, or JSON mode + Pydantic validation. Retry once on schema error with validator message. Log prompt hash, not prompt with PII.

SQL validation layer – mandatory, not optional

Generated SQL must pass all checks before execution. SELECT-only enforced via role + parser + policy layer. Syntax, operation, permission, cost, explain-plan.

VALIDATION CHECKLIST - EVERY QUERY MUST PASS

- ✓ **SELECT-only?** No INSERT/UPDATE/DELETE/DROP/ALTER/MERGE
- ✓ **Approved catalogs/views?** Only genie_semantic.*
- ✓ **No unauthorized columns** & hidden system tables
- ✓ **No comments / multi-stmt** — single statement only
- ✓ **Date filter + LIMIT** present for large tables
- ✓ **Join route approved?** No Cartesian join
- ✓ **Scan < threshold** — estimated bytes / rows + timeout
- ✓ **User row/col access?** Region, store, sensitive fields

⚠ Fail closed: any check fails → CLARIFICATION / ERROR with safe message, never raw DB error. Logged with policy_version + trace_id.

1 Parser & allowlist (sqlparse) — FAST • LOCAL

Parses AST, blocks forbidden keywords, validates only allowed views/columns/joins, rejects comments, ensures LIMIT & date filter.

2 Read-only warehouse — CREDENTIAL

Databricks SQL Warehouse with read-only service role, query timeout, max rows/bytes, concurrency limits. User OAuth where possible.

3 Unity Catalog permissions — GOVERNANCE

Row & column level security, sensitive-field exclusion, region/store scoping, lineage + audit. Final gate even if parser bypassed.

Execution – read-only, scoped, audited

After validation: Backend → Query Service → Databricks SQL Warehouse → Semantic Views. Supports stateful NL querying + programmatic integration (Genie Agents API pattern).

PRODUCTION FLOW

Backend → **Query Service** → SQL Warehouse →

Semantic Views

No direct DB access from LLM. Service account has SELECT-only role + timeout. Query runs under mapped user scope where possible.

QUERY SERVICE ENFORCES

- **Read-only** connection
- **Timeout** + max rows / bytes scanned
- **Result-size** limits + truncation check
- **Parameterized** filters — no injection
- **Concurrency** limits per user/team
- **Cancellation** + audit record

SAFER: USER OAUTH, NOT ONE SERVICE ACCOUNT

✓ User OAuth

Query runs with user-level or mapped role permissions. Store/region scoping via Unity Catalog. Different users see different rows — cache key must include security scope.

✕ One powerful SA

All users get same access = over-privileged, audit fails, cache leaks across security scopes. Never use.

OBSERVABILITY + AUDIT

Every execution logs: trace_id, sql_hash, dataset_version, policy_version, execution_time, bytes_scanned, row_count, timeout_flag, user_scope, warehouse_id. Enables cost control + lineage.

Read-only credential | **Result validator next**

Summarization – use only supplied rows

LLM explains result, not invents result. Input to summary model is validated evidence only. Prevents hallucination of causes or missing values.

INPUT TO SUMMARY MODEL – EVIDENCE ONLY

```
{
 "question": "Show sales by store in Bangkok last week",
 "metric_definition": "Net sales excl VAT",
 "sql": validated SQL,
 "rows": [
  {"store": "A", "sales": 5000000},
  {"store": "B", "sales": 4200000}
 ],
 "freshness": "2026-10-12T08:00:00+07:00"
}
```

🔒 **LLM never sees raw DB.** Only rows that passed validation + policy + row-level filter.

Output example – observation only

Store A had highest net sales at ฿5.0M, followed by Store B at ฿4.2M. Data refreshed at 08:00 Bangkok time.

Includes chart + table + freshness + dataset + assumptions + SQL if authorized.

SUMMARY PROMPT MUST SAY

- Use only supplied rows — do not invent causes
- Do not invent missing values or estimates
- Distinguish observation from explanation
- Mention incomplete or stale data explicitly

WHY THIS MATTERS FOR L6

Trust = **evidence + freshness + assumptions**, not fluent answer. If rows empty → say empty, not “no sales because…”. If truncated → warn. Explain failure modes.

Caching – never cache across security scopes

Use Redis or controlled cache, but cache key must include security scope + version. Never use only question as key — two users can have different permissions.

WHAT TO CACHE (4 LAYERS)

Semantic metadata
Views, metrics, joins, synonyms

Schema retrieval
Vector search results per question

Common aggregates
Short-lived query results

LLM responses
Non-sensitive identical prompts only

TTL GUIDANCE

METADATA	QUERY RESULTS	LLM RESPONSE
1 hour	5–15 min	30 min
semantics, schemas	freshness matters	non-sensitive only

Redis + semantic_version + data_freshness_version invalidation. On model or dataset version bump, flush relevant keys. Log cache hit/miss with trace_id.

CACHE KEY MUST INCLUDE — SECURITY AWARE

normalized_question · user_security_scope · semantic_version · data_freshness_version

⚠ **Anti-pattern:** key = md5(question). User A in Bangkok sees User B in Chiang Mai data. Row-level leak. Cross-scope cache = security incident.

Safe pattern: key = hash(normalized_q + role_hash + region_hash + store_scope + sem_ver + data_ver)

- Redis namespace per dataset
- PII never in cache value
- Short-lived results only — no long TTL for sensitive metrics
- Observability: cache hit rate by scope, not global

END-TO-END • AUDITED • VERSIONED

Complete production flow – 13 steps

1 User signs in SSO
Entra ID, roles, region, store scope

2 Frontend sends query
question + conversation_id + token

3 API Gateway
Validates token, rate-limits, trace_id

4 Backend auth + intent
Verify roles, classify: SQL / RAG / API

5 Metadata retriever
Relevant certified views + metrics only

6 LLM Gateway call 1
Question + controlled semantic context

7 LLM proposes SQL
Structured JSON + assumptions + 0.91 conf

8 SQL Policy Engine
Parser, allowlist, cost, permissions

9 Query Service executes
Read-only, User OAuth, timeout, LIMIT

10 Result validator
Empty? Abnormal? Truncated? Row caps

11 LLM Gateway call 2
Summarize evidence only, no invention

12 Frontend renders
Answer + chart + table + SQL + freshness

13 Observability
trace, model, tokens, cost, latency, feedback

VERSIONING & AUDIT

model_version + dataset_version + policy_version + feature_version + semantic_version + trace_id + user_scope — every recommendation replayable. Logs redact PII, hash SQL.

GOLDEN RULE

LLM reasons, backend authorizes. Two LLM calls only: SQL proposal + evidence summary. Validator between them cannot be skipped. Frontend untrusted.

Recommended retail stack — generic, governed, proven

No company names. Generic implementation you can run on Azure / AWS / Tencent. Front → Gateway → Orchestrator → Semantic Warehouse.

FRONTEND & ACCESS

- **Web:** React / Next.js + TypeScript, charting, SSE / WebSocket
- **Identity:** Entra ID / enterprise IdP, OAuth, corporate SSO
- **API control:** Azure APIM / Kong / Apigee / Tencent GW

TLS · Rate limit · WAF · trace_id

BACKEND & AI ORCHESTRATION

- **Backend:** Python FastAPI, Pydantic, deterministic state machine
- **Orchestration:** RECEIVED → ANSWER states, optionally LangGraph
- **LLM Gateway:** Provider abstraction, routing, budgets, redacted logging

GPT · Gemini · DeepSeek · Qwen

SEMANTIC & DATA PLANE

- **Metadata:** Unity Catalog + semantic registry (genie_semantic.*)
- **Retrieval:** Vector search over schemas, metrics, approved SQL
- **SQL:** Databricks SQL Warehouse — certified views only, read-only, User OAuth

Certified views enforce THB norm, standard dates, sensitive-field exclusion, last_updated freshness.

INFRA, CACHE & OBSERVABILITY

- **Cache:** Redis — metadata 1h, results 5-15m, LLM 30m, key = question + scope + version + freshness
- **Observability:** OpenTelemetry + logs + MLflow / eval store — trace, tokens, cost, policy
- **Deploy:** Docker + Kubernetes, secrets in Key Vault / cloud secret manager

What I would NOT do – production killers

Each anti-pattern is a security, cost, or trust failure. Avoiding them is what makes Text-to-SQL safe enough for enterprise retail data.

SECURITY KILLERS

- **Frontend calling LLM directly**
  Leaks API keys, bypasses auth, no audit
- **LLM holding DB credentials**
  Model never connects to production warehouse
- **One unrestricted SA for every user**
  Over-privileged, audit fails, cache leaks

DATA & COST KILLERS

- **LLM querying raw prod tables**
  1000s cols, sensitive fields, hallucinated joins
- **Sending entire catalog each prompt**
  Token bloat, cost, slower, less accurate
- **Allowing write SQL**
  INSERT/UPDATE/DELETE/DROP — must be SELECT-only

TRUST & COMPLIANCE KILLERS

- **Executing arbitrary SQL**
  No parser/allowlist/cost check → injection, scan bomb
- **Trusting fluent answer without evidence**
  Show chart/table/SQL/freshness/assumptions
- **Logging PII / caching across scopes**
  key ≠ question only, redact logs

Easy memory: User → API Gateway → Backend → Semantic Context → LLM Gateway → LLM proposes SQL → Validator approves → DB executes → LLM explains evidence. LLM reasons, deterministic services authorize/validate/execute.

If you do one thing right: Enforce 3-level validation (parser → read-only role → Unity Catalog) + User OAuth + structured JSON. Never let frontend be trusted.

CLOSING — PRODUCTION READY

Key principles for retail Text-to-SQL

1 LLM reasons & proposes, backend authorizes

Read-only assistant, human-in-the-loop for high-stakes. Deterministic services validate & execute — LLM never has DB creds.

2 Semantic layer is exclusive interface

Certified genie_semantic views only — business-friendly, THB normalized, versioned, sensitive fields excluded.

3 Retrieval, not full catalog dump

Vector search over schemas + metrics + approved SQL. Send only relevant context — cheaper, safer, accurate.

4 Structured output + 3-level validation

JSON contract → Parser/allowlist → Read-only warehouse → Unity Catalog row/col. SELECT-only, LIMIT, cost, timeout.

5 Versioning & auditability by design

model_version + feature_version + dataset_version + policy_version + trace_id + user_scope. Replayable, no PII in logs.

6 Success = adoption + evidence + trust

Monitor 4 layers: data quality, model quality, decision quality, business outcomes + cost. Answer with evidence, freshness, assumptions.

Easy memory: User → API GW → Backend → Semantic Context → LLM Gateway → LLM proposes SQL → Validator → Warehouse → LLM explains → User sees evidence.
Security = User OAuth + security-aware cache (question+scope+version) + never cache across scopes.